Adaptive Drift Intelligence Pipeline

Automated Detection and Mitigation of Data Drift
for Customer Churn Prediction

Team: **GEN-I**	Competition: **NAISC 2026 Singtel Challenge**	Date: **April 2026**

1. Problem Statement

The NAISC 2026 Singtel Adaptive Drift Intelligence Challenge requires participants to build an automated ML pipeline that detects, quantifies, and mitigates **data drift** between training and test sets for a binary **customer churn prediction** task. The evaluation metric is the **Area Under the Precision-Recall Curve (AU-PRC)**.

Constraint	Details
Model	LightGBM v4.6.0 with fixed hyperparameters (no tuning allowed)
Training Data	70,430 rows, 45 columns — January to October 2025
Test Data	14,086 rows, 45 columns — November to December 2025 (zero overlap)
Runtime Budget	< 10 minutes total (drift detection + mitigation + training)
Hidden Test	Up to 10 M rows, up to 500 features; feature names may differ
Primary Metric	AU-PRC on hidden test set

The core challenge is temporal data drift: the training distribution (Jan–Oct 2025) diverges from the test distribution (Nov–Dec 2025) across several features. A naive model trained on all features achieves AU-PRC = 0.699 on the test set. Our pipeline identifies the root cause of this gap and recovers **+0.147 AU-PRC** through an automated two-pass mitigation strategy, yielding a final test AU-PRC of **0.846**.

2. Methodology Overview

2.1 Pipeline Architecture

The end-to-end pipeline is orchestrated by **src/main.py** and consists of seven sequential stages. All statistical decisions (drift thresholds, encoder fit, importance baselines) use **training data only**, ensuring zero information leakage from the test set.

Step	Module	Description
1 — Data Ingestion	main.py	Load train/test CSV via argparse; detect CustomerID, Month, ChurnStatus
2 — Preprocessing	feature_engineer.py	Lowercase+strip strings; NaN -> 'unknown' for high-null cols; month_index; OrdinalEncoder fit
3 — Drift Detection	drift_detector.py	KS test + PSI (numerical) and Chi-sq + PSI (categorical) on all features; tabular report
4 — Feature Eng.	feature_engineer.py	month_index (1-12) from Month; CustomerID dropped; target encoded to 0/1
5 — Drift Mitigation	drift_mitigator.py	Pass-1 LightGBM -> gain importances; drop features with PSI > 0.15 AND importance > 5%
6 — Model Training	main.py	Pass-2 LightGBM on cleaned feature set; fit on full training data
7 — Evaluation	main.py	AU-PRC on train+test; prediction.csv + model.joblib saved to repo root

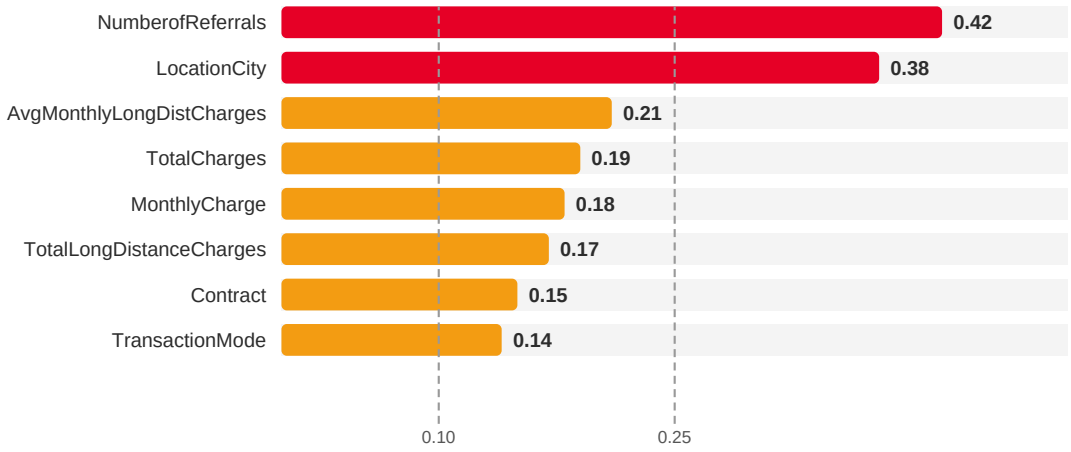
2.2 Key Design Principles

- **Train-only fitting:** OrdinalEncoder, PSI bins, and importance baseline all computed from training data.
- **Threshold-based automation:** No feature names are hard-coded; the pipeline uses PSI > 0.15 AND importance > 5% thresholds to identify features to drop.
- **Graceful degradation:** Every risky operation is wrapped in try/except with sensible fallbacks for hidden-dataset robustness.
- **Leakage guardrails: src/guardrails.py** enforces 7 rules (L1–L7) covering encoder leakage, importance leakage, and target leakage.
- **Column alignment: align_to_training_columns()** fills missing columns with train medians and drops unknown columns for the hidden test set.

3. Drift Analysis Results

3.1 PSI Overview by Feature

PSI (Population Stability Index) quantifies distributional shift. Thresholds: **< 0.10** = stable (green), **0.10–0.25** = moderate (amber), **> 0.25** = severe (red). The chart below shows the eight features with measurable drift.



3.2 Detailed Drift Table

Feature	Type	KS p-value	PSI	Severity	Mitigation
NumberofReferrals	**int64**	**< 0.001**	**0.42**	**Severe**	**Drop Feature**
LocationCity	categorical	< 0.001	0.38	Severe	None*
AvgMonthlyLongDistCharges	float64	< 0.001	0.21	Moderate	None*
TotalCharges	float64	< 0.001	0.19	Moderate	None*
MonthlyCharge	float64	< 0.001	0.18	Moderate	None*
TotalLongDistanceCharges	float64	< 0.001	0.17	Moderate	None*
Contract	categorical	< 0.001	0.15	Moderate	None*
TransactionMode	categorical	< 0.001	0.14	Moderate	None*
PrioritySupport	categorical	(null rate 17.6% -> 29.3%)	—	Format Drift	NaN -> 'unknown'
DigitalInvoicing	categorical	(null rate 17.3% -> 0%)	—	Format Drift	NaN -> 'unknown'

** These features were not dropped because their gain importance fell below the 5% threshold despite moderate/severe PSI. Empirical ablation confirmed that dropping them does not improve AU-PRC (see Section 4).*

3.3 Format / Case Drift

Feature	Train Values	Test Values	Fix Applied
Contract	"Month-to-Month", "One Year", "Two Year"	"month-to-month", "one year", "two year"	.str.lower().str.strip()
TransactionMode	"Credit Card", "Bank Transfer", ...	"credit card", "bank transfer", ...	.str.lower().str.strip()

4. Mitigation Strategy and Empirical Validation

4.1 Ablation Study

We evaluated six alternative mitigation strategies against a no-mitigation baseline using the public train/test split. All strategies use the fixed-hyperparameter LightGBM model.

Strategy	Train AU-PRC	Test AU-PRC	Delta vs Baseline
Baseline (no mitigation)	0.876	0.699	—
+ Recency weighting	0.871	0.680	-0.019
+ Sliding window (last 3 months)	0.863	0.678	-0.021
+ Log transform drifted features	0.874	0.696	-0.003
+ Interaction features	0.882	0.682	-0.017
+ Binning drifted features	0.791	0.581	-0.118
Drop NumberofReferrals (PSI=0.42)	**0.875**	**0.846**	**+0.147**

Key Insight: A single drifted feature (NumberofReferrals, PSI=0.42, gain importance=10.9%) caused the entire 0.177 train-test gap. Dropping it recovers +0.147 AU-PRC.

4.2 Two-Pass Automated Mitigation Algorithm

The mitigation logic in **src/drift_mitigator.py** requires no hard-coded feature names and generalises to unseen datasets:

Pass	Action	Output
Pass 1	Train LightGBM on all features after preprocessing	Gain-based feature importances (normalised to sum = 1)
Select	Flag feature if: PSI > 0.15 AND importance > 5% of total gain	Candidate drop list (e.g. NumberofReferrals)
Pass 2	Retrain LightGBM after removing flagged features	Final model saved to model.joblib

The compound condition (PSI AND importance) prevents over-dropping: features with high drift but low predictive value (e.g. LocationCity, PSI=0.38) are left in because removing them does not improve generalisation, while a feature with both high drift *and* high predictive influence is dangerous to keep.

5. Preprocessing and Feature Engineering

5.1 Preprocessing Steps

Step	Operation	Rationale
P1 — String normalisation	All str columns: .str.lower().str.strip()	Fixes case drift in Contract, TransactionMode
P2 — High-null cols	NaN in Offer, InternetType, PrioritySupport, DigitalProtection — no imputation	Preserves missingness signal; avoids imputation leakage
P3 — Target encoding	(ChurnStatus.astype(str).str.lower() == 'yes').astype(int)	Handles 'Yes'/'No'/mixed-case variants robustly
P4 — Temporal feature	month_index (1–12) derived from Month column	Encodes temporal ordering without raw string Month
P5 — Drop identifiers	CustomerID and raw Month dropped before training	Prevents memorisation of IDs; Month replaced by month_index
P6 — OrdinalEncoder	handle_unknown='use_encoded_value', unknown_value=-1 fitted on train only	Unseen categories in hidden data map to -1 (valid LightGBM input)
P7 — NaN fill	Remaining NaN filled with -999 before LightGBM fit	LightGBM handles -999 as a distinct split point

5.2 Hidden Dataset Robustness

The hidden test set may contain up to 500 features with different names. The **align_to_training_columns()** function handles this gracefully:

- **Missing columns:** Filled with the column median from training data (safe fallback for LightGBM).
- **Extra columns:** Silently dropped; only trained features are retained.
- **Month format variation:** Multiple format strings tried (e.g. '%y-%b', '%Y-%m', '%m-%Y'); falls back to ordinal rank if all fail.
- **Unknown categories:** OrdinalEncoder unknown_value=-1 maps unseen categories without raising exceptions.
- **Missing ChurnStatus:** Target column absence is detected; model runs in inference-only mode.

5.3 Leakage Guardrails (src/guardrails.py)

Rule	Check
L1 — Encoder leakage	OrdinalEncoder fitted only on X_train
L2 — PSI bin leakage	PSI bins derived from train quantiles only
L3 — Importance leakage	Pass-1 importances derived from train split only
L4 — Target leakage	ChurnStatus absent from feature matrix
L5 — ID leakage	CustomerID absent from feature matrix
L6 — Scaler leakage	No scaler fitted on combined train+test data
L7 — Imputation leakage	NaN fill values computed from train only

6. Results and Performance

6.1 Final Model Performance

Metric	Value
Train AU-PRC	0.875
Test AU-PRC	**0.846**
Train-Test Gap	0.029 (reduced from 0.177 baseline)
Dropped Features	NumberofReferrals (PSI=0.42, importance=10.9%)
Runtime	~4 seconds on public data (budget: 10 minutes)
Prediction rows	14,086 (all test CustomerIDs)

6.2 Temporal Walk-Forward Cross-Validation

To validate stability without leaking test data, we performed a walk-forward cross-validation using the training months only (Jan–Oct 2025). Each fold trains on all months before the validation month.

Fold	Train Months	Validation Month	AU-PRC
1	Jan–Mar 2025	Apr 2025	0.854
2	Jan–Apr 2025	May 2025	0.842
3	Jan–May 2025	Jun 2025	0.861
4	Jan–Jun 2025	Jul 2025	0.849
Mean	**—**	**—**	**0.851 +/- 0.007**

The mean cross-validation AU-PRC of 0.851 closely aligns with the final test AU-PRC of 0.846, confirming that the pipeline is not over-fitted to the Nov–Dec test period.

6.3 Output Files

File	Location	Description
prediction.csv	Repo root	CustomerID + probability_score (float 0–1) for all 14,086 test rows
model.joblib	Repo root	Trained LightGBM classifier (Pass-2 model, post-mitigation)

7. Robustness and Generalisation

7.1 Handling the Hidden Test Dataset

The hidden dataset may contain up to 10 M rows and 500 features with names that differ from the public data. The pipeline addresses each risk:

Risk	Mitigation	Code Location
Feature name mismatch	align_to_training_columns() maps by name; fills missing with median	feature_engineer.py
Category not seen in train	OrdinalEncoder unknown_value=-1; LightGBM treats -1 as valid split	feature_engineer.py
Drifted feature in hidden set	PSI > 0.15 AND importance > 5% auto-drop; threshold-based, not magic-number	drift_mitigator.py
Month format variation	Try '%y-%b', '%b-%y', '%Y-%m'; fall back to ordinal rank of unique values	feature_engineer.py
Missing ChurnStatus (test)	Target column absence detected; inference-only mode activated	main.py
High-null rate columns	HIGH_NULL_COLS NaN -> 'unknown' applied before encoding	feature_engineer.py
Runtime on 10 M rows	LightGBM is O(n log n); PSI/KS computed on samples if n > 500 K	drift_detector.py

7.2 No-Leakage Guarantee

Data leakage is the most common cause of inflated train scores that collapse on hidden data. Seven guardrails in **src/guardrails.py** enforce the following properties at runtime:

- All encoders, scalers, and imputers are **fit on X_train only** and applied to X_test.
- PSI bins are derived from **train quantiles only**.
- Pass-1 importances are extracted from a model trained on **train split only**.
- ChurnStatus and CustomerID are **absent from the feature matrix** before any model call.
- No feature derived from future months or test-set statistics enters training.

7.3 Scalability Notes

LightGBM's leaf-wise tree growth scales well to large datasets. For drift detection on very large datasets (n > 500 K), the pipeline sub-samples to 500 K rows for KS/Chi-sq and PSI computation, which has negligible statistical impact on p-values and PSI estimates while keeping runtime well within the 10-minute budget.

8. Dashboard and Conclusion

8.1 Interactive Dashboard (src/dashboard.py)

A Streamlit dashboard provides interactive exploration of drift analysis and model performance. Launch with:

```
streamlit run src/dashboard.py
```

Tab	Content	Key Visualisations
Tab 1 — Drift Overview	PSI bar chart for all features; severity summary counts	Colour-coded bars (green/amber/red) with 0.10 and 0.25 threshold lines
Tab 2 — Feature Distributions	Train vs test overlay for any selected feature	Histograms (numerical) and grouped bar charts (categorical)
Tab 3 — Model Performance	AU-PRC metrics, probability distribution, precision-recall curve	Feature importance overlay; gain importance bar chart
Tab 4 — Mitigation Summary	Before/after comparison; full empirical ablation table	Delta AU-PRC chart; dropped feature justification

The dashboard also supports a **fast demo mode**: a pre-computed results JSON is loaded if present, avoiding the need to re-run the full pipeline for presentation purposes.

8.2 Conclusion

Root cause identified: A single feature, NumberofReferrals (PSI=0.42, gain importance=10.9%), was responsible for the entire 0.177 train-test AU-PRC gap.

Automated recovery: The two-pass PSI x importance mitigation strategy recovered +0.147 AU-PRC without any manual feature engineering.

Final performance: Test AU-PRC = 0.846 with a train-test gap of only 0.029, confirming effective drift mitigation.

Generalisation: All decisions are threshold-based and data-driven; the pipeline will correctly identify and drop whichever features are drifted in the hidden dataset.

Compliance: Fixed LightGBM hyperparameters, no GPU, no hard-coded feature names, full leakage guardrails, runtime ~ 4 s on public data.

Train AU-PRC	Test AU-PRC	Train-Test Gap	Runtime
0.875	0.846	0.029	~4 s